计算机系统基础实验报告

Lab 1：Data Lab

姓名：陈璋睿　学号：23300160006

远程仓库：`https://github.com/ritachen3656251-design/ICS_lab1`

一、实验目的与任务

本实验完成 `bits.c` 中的 19 个函数，内容包括补码、掩码、移位、整数溢出以及 IEEE 754 单精度浮点数的编码和舍入。实现时需要同时满足函数语义与逐题操作数上限 [1]。

整数部分以 32 位补码为基础，使用算术右移，并按题设保留运算结果的低 32 位。报告先说明各函数的实现，再介绍三个函数的适度优化，最后给出实际运行的检查结果。当前版本的 19 个函数均通过规则检查，`btest` 得分为 **110/110**。

二、实验环境与构建方法

项目	配置
本机系统	Ubuntu 22.04.5 LTS
本机体系结构	x86_64
编译器	GCC 11.4.0
编译选项	`-O -Wall -fwrapv -m32`
规则检查	`./check_ops.py bits.c`
正确性测试	`./btest`

先构建程序，再分别运行规则检查和正确性测试：

```
make all
./check_ops.py bits.c
./btest
```

`check_ops.py` 检查每题使用的运算符及其数量；`btest` 将实现结果与参考行为比较。也可以通过 `./test.sh` 完成重新构建和两项检查。

三、整数函数的实现

P1 signMask

将常量 1 左移 31 位，使第 31 位为 1，其余位为 0，得到 0x80000000。

signMask

```
int signMask(void) {
  return 1 << 31;
}
```

P2 bitXor

异或在两位不同时为 1。用德摩根律把“至少一位为 1”写成 ~(~x & ~y)，再与“不能同时为 1”的 ~(x & y) 相与，整个表达式只使用 ~ 和 &。

bitXor

```
int bitXor(int x, int y) {
  return ~(~x & ~y) & ~(x & y);
}
```

P3 negativePart

x >> 31 在非负数时产生全零掩码，在负数时产生全一掩码。将 ~x + 1 与该掩码相与，即可在负数时返回补码取负的结果，否则返回零。INT_MIN 的结果按题设保留原来的位模式。

negativePart

```
int negativePart(int x){
  int negativeMask = x >> 31;
  return (~x + 1) & negativeMask;
}
```

P4 copyByteWithin

字节编号乘 8 得到移位量。先右移并与 255 相与，取出原数中的源字节；随后清空目标字节，再把源字节移入。即使源、目标编号相同，也会得到原数。

copyByteWithin

```
int copyByteWithin(int x, int src, int dst) {
  int sourceShift = src << 3;
  int targetShift = dst << 3;
  int sourceByte = (x >> sourceShift) & 255;
  int targetMask = 255 << targetShift;
  return (x & ~targetMask) | (sourceByte << targetShift);
}
```

P5 `logicalShift`

算术右移会复制符号位，因此需要在右移后清除高位补入的 1。掩码由最高位的 1 右移、再左移一位并取反得到。分成两次移位后，`n=0` 时不会出现移位 32 位的表达式。

logicalShift

```
int logicalShift(int x, int n) {
  int retainedMask = ~(((1 << 31) >> n) << 1);
  return (x >> n) & retainedMask;
}
```

P6 `swapNibblePairs`

构造重复的 `0x0f` 掩码，分别提取每个字节的低四位与高四位。低四位左移 4 位，高四位右移 4 位并清除其他位，两部分按位或后就完成四组交换。

swapNibblePairs

```
int swapNibblePairs(int x) {
  int lowNibbleMask = 15 | (15 << 8);
  lowNibbleMask = lowNibbleMask | (lowNibbleMask << 16);
  return ((x & lowNibbleMask) << 4) | ((x >> 4) & lowNibbleMask);
}
```

P7 `secondLowestZeroBit`

`x + 1` 将最低的零置为一，并使它右侧的连续一变为零；再与原数按位或，就只填上了最低的零。对得到的 `filled` 使用 `~filled & (filled + 1)`，即可定位下一个零。原数不足两个零时，结果为零。

secondLowestZeroBit

```
int secondLowestZeroBit(int x) {
  int filled = x | (x + 1);
  return ~filled & (filled + 1);
}
```

P8 `oddParity`

依次按 16、8、4、2、1 位进行异或折叠，最终最低位表示原数中 1 的个数的奇偶性。题目要求的是补成奇校验的位，因此最低位为零时返回 1，为一时返回 0。

oddParity

```
int oddParity(int x) {
  x = x ^ (x >> 16);
  x = x ^ (x >> 8);
  x = x ^ (x >> 4);
  x = x ^ (x >> 2);
  x = x ^ (x >> 1);
  return !(x & 1);
}
```

P9 `rotateRightBits`

先用 `n & 31` 将旋转量化为 0 到 31。右移部分使用掩码实现逻辑右移，另一部分把移出的低位移到高位，再将两部分合并。反向移位量也对 32 取模；旋转零位时，两部分都等于原数。

rotateRightBits

```
int rotateRightBits(int x, int n) {
  int shift = n & 31;
  int backShift = (~shift + 1) & 31;
  int retainedMask = ~(((1 << 31) >> shift) << 1);
  return ((x >> shift) & retainedMask) | (x << backShift);
}
```

P10 `roundEvenPow2`

设 $x = q2^n + r$，其中 $0 \le r < 2^n$。在右移取商前给 x 加上

$$2^{n-1} - 1 + (q \,\&\, 1).$$

余数小于半程时商不变，大于半程时商加一；余数恰好为半程时，只有原商为奇数才进位。最后左移 n 位，恢复为 2^n 的倍数。例如 10 与 14 分别舍入到最近的 4 的倍数时，结果为 8 与 16。

roundEvenPow2

```
int roundEvenPow2(int x, int n) {
  int half = 1 << (n + ~0);
  int quotientOdd = (x >> n) & 1;
  int biased = x + half + ~0 + quotientOdd;
  return (biased >> n) << n;
}
```

P11 `midpointTowardFirst`

先用

$$\left\lfloor \frac{x+y}{2} \right\rfloor = (x \,\&\, y) + ((x \oplus y) \gg 1)$$

计算向下取整的中点，避免直接计算可能溢出的 $x + y$。当两数之和为奇数且 $x > y$ 时，中点还需加一，才能向第一个参数靠拢。比较大小时，异号输入根据符号判断，同号输入根据 $y - x$ 的符号判断。

midpointTowardFirst

```
int midpointTowardFirst(int x, int y) {
  int differingBits = x ^ y;
  int lowerMidpoint = (x & y) + (differingBits >> 1);
  int reverseDifference = y + (~x + 1);
  int firstGreater = ((differingBits & ~x) |
                      (~differingBits & reverseDifference)) >> 31;
  return lowerMidpoint + (differingBits & firstGreater & 1);
}
```

P12 `isBetweenEitherOrder`

分别计算 $x < a$ 与 $x < b$ 的结果。当两者不同时，x 位于两个端点之间；再补上 $x = a$ 或 $x = b$ 的情况，就得到闭区间判断。每次比较都按是否异号选择依据：异号时看 x 的符号，同号时看差值的符号。用于判断异号的异或结果也可以直接用于判断相等。

isBetweenEitherOrder

```
int isBetweenEitherOrder(int x, int a, int b) {
  int differentA = x ^ a;
  int differentB = x ^ b;
  int belowA = ((differentA & x) |
                (~differentA & (x + (~a + 1)))) >> 31;
  int belowB = ((differentB & x) |
                (~differentB & (x + (~b + 1)))) >> 31;
  return ((belowA ^ belowB) & 1) | !differentA | !differentB;
}
```

P13 `mul5Sat`

通过加法依次求 $2x$、$4x$ 和 $5x$，并检查各阶段的符号变化。只比较 x 与最终结果的符号不足以判断溢出，因为多次回绕后可能恢复原符号。发生溢出时，按原输入的符号选择 `INT_MAX` 或 `INT_MIN`；否则保留计算出的五倍值。

mul5Sat

```
int mul5Sat(int x) {
  int twice = x + x;
  int fourTimes = twice + twice;
  int fiveTimes = fourTimes + x;
  int overflowMask = ((x ^ twice) | (twice ^ fourTimes) |
                      (fourTimes ^ fiveTimes)) >> 31;
  int saturation = (1 << 31) ^ ~(x >> 31);
  return fiveTimes ^ ((fiveTimes ^ saturation) & overflowMask);
}
```

P14 `classifyAdd3`

将每个输入精确写成 $x = 4q_x + r_x$，其中 $q_x = x \gg 2$，$r_x = x \,\&\, 3$。令

$$H = q_x + q_y + q_z + \left\lfloor \frac{r_x + r_y + r_z}{4} \right\rfloor .$$

三个商与余数进位之和仍可用 32 位有符号整数表示。原数学和可表示，当且仅当 H 位于 $[-2^{29}, 2^{29} - 1]$，即 `H >> 29` 全为符号位。超出该范围时，根据 H 的符号返回 1 或 -1，否则返回零。

classifyAdd3

```
int classifyAdd3(int x, int y, int z) {
  int lowSum = (x & 3) + (y & 3) + (z & 3);
  int highSum = (x >> 2) + (y >> 2) + (z >> 2) + (lowSum >> 2);
  int sign = highSum >> 31;
  int overflow = !!((highSum >> 29) ^ sign);
  return (sign | 1) & (~overflow + 1);
}
```

四、浮点编码与舍入

P15 `floatScaleThreeHalves`

单精度数包含 1 位符号、8 位阶码和 23 位小数。规格化数的有效数还包含隐藏的最高位 1；非规格化数的隐藏位为 0[3]。先拆出这些部分，在整数域将有效数精确乘三，最后统一除以二并舍入，避免中间舍入丢失低位。

对规格化数，若三倍有效数达到 `0x2000000`，需要右移两位并相应增加阶码；否则右移一位。非规格化数按最小非规格化单位计算，越过规格化边界时由编码加法完成进位。丢弃部分大于半程时进位；等于半程时，仅在保留部分为奇数时进位。无穷和 NaN 原样返回，溢出结果置为同符号无穷。

floatScaleThreeHalves

```c
unsigned floatScaleThreeHalves(unsigned uf) {
  unsigned sign = uf & 0x80000000u;
  unsigned exponent = (uf >> 23) & 255;
  unsigned significand = uf & 0x7fffff;
  unsigned triple;
  unsigned rounded;
  unsigned remainder;
  unsigned half;
  unsigned magnitude;
  int shift = 1;
  if (exponent == 255)
    return uf;
  if (exponent != 0)
    significand = significand | 0x800000;
  triple = significand * 3;
  if (exponent != 0) {
    if (triple >= 0x2000000) {
      shift = 2;
      exponent = exponent + 1;
    }
    exponent = exponent - 1;
  }
  rounded = triple >> shift;
  remainder = triple & ((1u << shift) - 1);
  half = 1u << (shift - 1);
  if (remainder > half || (remainder == half && (rounded & 1)))
    rounded = rounded + 1;
  magnitude = (exponent << 23) + rounded;
  if (magnitude >= 0x7f800000)
    magnitude = 0x7f800000;
  return sign | magnitude;
}
```

P16 `floatRoundEven`

按阶码将输入分为三个区间。绝对值不小于 2^{23} 的有限数本身已经是整数，同一分支也原样返回无穷与 NaN；绝对值小于 0.5 时返回同符号零；位于 $[0.5, 1)$ 时，恰好 0.5 取零，其余取 1，并保留符号。

其余输入的阶码记为 E，编码中的一个整数单位为 `1u << (150 - exponent)`。用这个单位构造丢弃位掩码，得到整数部分和余数，再按“最近、相等时取偶数”判断是否增加一个单位。最终重新并入符号位。例如 $1.5 \to 2$、$2.5 \to 2$、$-0.5 \to -0$。

floatRoundEven

```
unsigned floatRoundEven(unsigned uf) {
  unsigned sign = uf & 0x80000000u;
  unsigned magnitude = uf & 0x7fffffffu;
  unsigned exponent = magnitude >> 23;
  unsigned unit;
  unsigned discardedMask;
  unsigned remainder;
  unsigned half;
  unsigned rounded;
  if (exponent >= 150)
    return uf;
  if (exponent < 126)
    return sign;
  if (exponent == 126) {
    if (magnitude == 0x3f000000)
      return sign;
    return sign | 0x3f800000;
  }
  unit = 1u << (150 - exponent);
  discardedMask = unit - 1;
  remainder = magnitude & discardedMask;
  half = unit >> 1;
  rounded = magnitude & ~discardedMask;
  if (remainder > half ||
      (remainder == half && ((magnitude | 0x800000) & unit)))
    rounded = rounded + unit;
  return sign | rounded;
}
```

P17 `float_i2f`

先记录符号，并在无符号整数中保存绝对值，这样 `INT_MIN` 的大小 2^{31} 也可以表示。随后通过右移查找最高的 1 所在的位置 h，由此确定浮点数的阶码。

若 $h \le 23$，有效位不超过 24 位，直接左移对齐即可精确表示。否则右移 $h-23$ 位，并根据丢弃位执行最近偶数舍入。组装结果时将阶码预减一，再加上含隐藏位的有效数；如果舍入使有效数进位，编码加法会同时推进阶码。例如 $2^{24}+1$ 恰好处于两个可表示值的中间，应舍入为 2^{24}。

float_i2f

```c
unsigned float_i2f(int x) {
  unsigned sign = 0;
  unsigned magnitude = x;
  unsigned scan;
  unsigned significand;
  unsigned remainder;
  unsigned half;
  int highestBit = 0;
  int shift;
  if (x == 0)
    return 0;
  if (x < 0) {
    sign = 0x80000000u;
    magnitude = 0u - magnitude;
  }
  scan = magnitude;
  while (scan >> 1) {
    scan = scan >> 1;
    highestBit = highestBit + 1;
  }
  if (highestBit <= 23) {
    significand = magnitude << (23 - highestBit);
  } else {
    shift = highestBit - 23;
    significand = magnitude >> shift;
    remainder = magnitude & ((1u << shift) - 1);
    half = 1u << (shift - 1);
    if (remainder > half || (remainder == half && (significand & 1)))
      significand = significand + 1;
  }
  return sign | (((highestBit + 126) << 23) + significand);
}
```

五、并行计数与位反转

P18 `bitCount`

采用分组累加：先统计每两位中 1 的个数，再合并为每四位、每八位的计数，最后将各字节计数相加。每一级的分组都足够容纳对应计数，例如一个字节最多包含 8 个 1。总数最大为 32，因此最终只需保留低六位。

掩码依次为重复的 `0x55`、`0x33` 和 `0x0f`。当前实现由交替字节掩码逐级移位异或得到它们，减少了分别拼接各掩码的操作数。

bitCount

```
int bitCount(int x) {
  int alternatingBytes = 255 | (255 << 16);
  int byteMask = alternatingBytes ^ (alternatingBytes << 4);
  int nibbleMask = byteMask ^ (byteMask << 2);
  int pairMask = nibbleMask ^ (nibbleMask << 1);
  x = (x & pairMask) + ((x >> 1) & pairMask);
  x = (x & nibbleMask) + ((x >> 2) & nibbleMask);
  x = (x + (x >> 4)) & byteMask;
  x = x + (x >> 8);
  x = x + (x >> 16);
  return x & 63;
}
```

P19 `bitReverse`

按分治方式交换相邻的单个位、两位组和半字节。完成三轮交换后，每个字节内部已反转，再直接颠倒四个字节的位置，就得到整个 32 位数的反转结果。

掩码从 `0x00ff00ff` 出发，通过左移后异或，依次形成 `0x0f0f0f0f`、`0x33333333` 和 `0x55555555`。最后把字节交换直接合并到返回表达式，当前共使用 34 次操作。

bitReverse

```
int bitReverse(int x)
{
  int byteMask = 255 | (255 << 16);
  int nibbleMask = byteMask ^ (byteMask << 4);
  int pairMask = nibbleMask ^ (nibbleMask << 2);
  int bitMask = pairMask ^ (pairMask << 1);
  int middleByteMask = 255 << 8;
  x = ((x >> 1) & bitMask) | ((x & bitMask) << 1);
  x = ((x >> 2) & pairMask) | ((x & pairMask) << 2);
  x = ((x >> 4) & nibbleMask) | ((x & nibbleMask) << 4);
  return (x << 24) | ((x & middleByteMask) << 8) |
         ((x >> 8) & middleByteMask) | ((x >> 24) & 255);
}
```

六、优化过程与新旧对比

在保存已通过全部测试的版本后，对区间判断、饱和乘法和位计数做了三处修改。下表的操作数来自课程检查器。

函数	优化前	优化后	上限
`isBetweenEitherOrder`	26	24	48
`mul5Sat`	17	16	30
`bitCount`	28	24	40

6.1 复用区间判断中的异或结果

`differentA` 和 `differentB` 已分别保存 `x ^ a`、`x ^ b`，相等判断直接对这两个变量取逻辑非即可，无需再次计算异或。因此减少两次操作，算法本身不变。

isBetweenEitherOrder

```
// main
  return ((belowA ^ belowB) & 1) | !(x ^ a) | !(x ^ b);

// optimize
  return ((belowA ^ belowB) & 1) | !differentA | !differentB;
```

6.2 简化饱和结果的掩码选择

参考资料中的按掩码合并位的恒等式 [2]，将四次操作的选择表达式改为三次操作。掩码为零时，异或项为零，返回 `fiveTimes`；掩码为全一时，两个 `fiveTimes` 互相抵消，返回 `saturation`。

mul5Sat

```
// main
  return (overflowMask & saturation) | (~overflowMask & fiveTimes);

// optimize
  return fiveTimes ^ ((fiveTimes ^ saturation) & overflowMask);
```

6.3 共用位计数的掩码构造过程

并行位计数资料说明了逐层合并小组计数的思路 [2]。本次保留原有累加过程，只把三个独立掩码的构造改为已有 `bitReverse` 中的逐级构造：从交替字节生成半字节掩码，再生成两位组与单个位掩码。构造部分由 12 次操作降为 8 次。

bitCount

```
// main
  int pairMask = 85 | (85 << 8);
  int nibbleMask = 51 | (51 << 8);
  int byteMask = 15 | (15 << 8);
  pairMask = pairMask | (pairMask << 16);
  nibbleMask = nibbleMask | (nibbleMask << 16);
  byteMask = byteMask | (byteMask << 16);

// optimize
  int alternatingBytes = 255 | (255 << 16);
  int byteMask = alternatingBytes ^ (alternatingBytes << 4);
  int nibbleMask = byteMask ^ (byteMask << 2);
  int pairMask = nibbleMask ^ (nibbleMask << 1);
```

三处修改合计减少 7 次源代码操作。

七、实验体会

本实验中，同一组位既可以表示数值，也可以作为选择条件。符号扩展能够生成全零或全一掩码，异或能够表示差异，移位则同时承担位置调整和分组处理的作用。把运算拆成这些基本步骤后，许多原本需要分支的判断可以写成直接的位表达式。

整数题需要结合范围理解中间结果，例如中点计算先分离公共位，三数求和先拆成商与余数。浮点题则需要区分编码与数值：隐藏位参与有效数计算，阶码决定小数位的位置，舍入还可能引起新的进位。优化过程中，复用已有结果与减少掩码构造，比单纯缩短代码行数更直接。

八、测试结果

本次对报告对应版本实际执行规则检查和 `btest`。下面两图是在终端中运行命令后截取的结果，原始输出同时保存在报告的 `assets/` 目录中。19 个函数的规则检查全部通过，正确性测试各项错误数均为零，总分为 **110/110**。

```
$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 16/32 operators
isBetweenEitherOrder: 24/48 operators
mul5Sat: 16/30 operators
classifyAdd3: 21/52 operators
floatScaleThreeHalves: 28/60 operators
floatRoundEven: 23/65 operators
float_i2f: 26/40 operators
bitCount: 24/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```
$ ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```

参考文献

[1] 复旦大学计算机系统基础课程组. Lab 1: Data Lab 实验说明 [EB/OL]. https://ics-26fall-fdu.github.io/labs/lab1-data-lab/. 访问日期：2026-10-03. 本地配套说明：项目 README.md 与 bits.c 中的题目要求。

[2] Sean Eron Anderson. Bit Twiddling Hacks[EB/OL]. https://graphics.stanford.edu/~seander/bithacks.html. 访问日期：2026-10-04. 本报告使用其中 “Merge bits from two values according to a mask” 的位选择恒等式，并参考 “Counting bits set, in parallel” 的分组计数思路。

[3] Sun Microsystems. Numerical Computation Guide, Chapter 2: IEEE Arithmetic[EB/OL]. Oracle 文档存档. https://docs.oracle.com/cd/E19059-01/stud.10/819-0499/ncg_math.html. 访问日期：2026-10-04. 用于核对单精度字段、规格化数的隐藏位和最近偶数舍入规则。